

IN2033 IMPLEMENTATION REPORT

LEON SEFERI

(EVERYTHING NEEDED FROM THE CODE AND TESTING FOR FINAL IMPLEMENTATION REPORT)

*Section numbering is preliminary and subject to change according to the other content needed in Implementation report.

*Will be compiled by another team member, forming our final report.

SECTION 5.1 – SUBSYSTEM SET UP

IPOS-SA is implemented as a two-part web application, consisting of a REST API backend and single page application frontend. These are both designed to run locally on a single machine.

Backend – Python / FastAPI

The backend is written in Python 3.14+ using the FastAPI framework. Python is an interpreted language so requires no compilation step. Furthermore, the application is managed using uv, a Python package manager. This automatically resolves and installs all dependencies on first run.

The server can be started for the first time using the following:

cd backend

pip install uv

uv sync

uv run python seed_data_demo.py # populate with demo data

uv run uvicorn main:app --reload

The Uvicorn ASGI server (with FastAPI's standard install) handles all the HTTP requests. The database is SQLite in development – a single file (ipos_sa.db) is created automatically in the backend directory on the first run.

PostgreSQL is supported for production through the DATABASE_URL environment variable.

Frontend – JavaScript/React/Vite

The frontend is written in JavaScript (ES modules) using the React 19 library. It is served using Vite 7 – a development server and build tool. Additionally, Node.js v20 or higher is needed.

The frontend can be started for the first time using:

cd frontend

npm install

npm run dev

*Please note, to start application, open two terminals and start both backend and frontend.

The application runs on <http://localhost:5173>

For production deployment, npm run build bundles the frontend into static files in frontend/dist/.

SECTION 5.1.1 – FILES CREATED

Python is an interpreted language so no compiled binary files (.exe, .jar, .class) are produced. The runtime consists of source files executed directly by the Python interpreter. The equivalent of a compiled executable is the combination of the source files and virtual environment installed by uv.

File / Directory	Description	Location at Runtime
main.py	Entry point for application. Initialises the FastAPI app and registers all routers. It also configures CORS and creates all of the database tables upon startup.	backend/
lpos_sa.db	SQLite database file that is created automatically on first run, when running seed data script. Contains the users, orders, products and invoices.	backend/

.venv/	Python virtual environment containing all installed packages (FastAPI, SQLAlchemy, PyJWT, bcrypt). Created and managed by uv.	backend/.venv/
auth/, catalogue/, merchants/, orders/, reports/, commercial_applications/, audit/, core/	Feature modules where each contains models.py, router.py and service.py. These are executed by the Python Interpreter at runtime.	backend/ <module>/
seed_data_demo.py	Script for populating the database with the official IPOS sample dataset prior to the demo. Run before starting backend.	backend/
frontend/dist/ (production only)	Static frontend files produced by npm run build. Contains index.html and hashed JS/CSS assets. Only needed if the Vite dev server is not used.	frontend/dist/

In development and demo mode, the Vite dev server serves the frontend directly from source so no build step is required.

SECTION 6.1 – COMMERCIAL OFF-THE-SHELF SOFTWARE COMPONENTS

Backend runtime dependencies that are installed automatically by uv.

Component	Version	Purpose
Python	3.14+	Interpreter / runtime environment.
FastAPI	>= 0.128.5	Rest API web framework.
Uvicorn	Standard – Bundled with FastAPI	HTTP Server.
SQLModel	>= 0.0.32	ORM – database models and queries. Built on SQLAlchemy + Pydantic.

Pydantic	>= 2.12.5	Request/response data validation and serialisation.
Pydantic-settings	>= 2.12.0	Configuration management via environment variables.
PyJWT	>= 2.11.0	JWT token generation and validation for authentication.
bcrypt	>= 5.0.0	Password hashing.
requests	>= 2.32.5	HTTP client for outbound calls to IPOS-PU.

Frontend runtime dependencies installed by npm install.

Component	Version	Purpose
Node.js	V20+	JavaScript runtime / package manager host.
React	19.2.0	UI component library.
React DOM	19.2.0	DOM rendering for React.
React Router DOM	7.13.0	Client side routing between pages.
React Icons	5.5.0	Icon library (Feather icons used throughout).
Vite	7.3.1	Deployment server and production build tool.

SECTION 6.2 – DEPLOYMENT OF RUNTIME FILES

All runtime components of IPOS-SA are deployed on the same computer. No remote database server or remote application server is used. The SQLite database is a local file within the project directory. Both the backend API (localhost:8000) and the frontend (localhost:5173) run as local processes on the same machine.

The other IPOS subsystems (IPOS-CA and IPOS-PU) are also deployed on the same machine for integration testing purposes.

IPOS-SA communicates with IPOS-PU via a REST endpoint for membership application notifications.

IPOS-CA communicates with IPOS-SA via the IPOS-SA REST API.

All inter subsystem communication occurs over localhosts.

In conclusion, all runtime components of IPOS-SA (dependencies, database and other IPOS subsystems) are deployed on the same computer.

SECTION 7.1 – UNIT TESTING OF IPOS-SA

Unit tests are written using Vilest (the JavaScript equivalent of junit). Tests are located in tests/services/ and run with:

```
cd frontend
```

```
npm test -- --run
```

The apiClient is mocked in all unit tests so that no running backend is required.

Two service modules (the JavaScript equivalent of classes) are covered. orderService.js implements OrderAPIImpl while authService.js implements LoginAPIImpl and staff account management.

orderService.js (OrderAPIImpl – Test Cases 1-27)

This wraps all order and catalogue API calls and exposes 6 functions (**placeOrder**, **trackOrderProgress**, **getOrderDetails**, **queryBalance**, **viewPreviousOrders**, **getCatalogue**). These are consistent with the class diagram generated in the first deliverable.

*Test Case IDs are references to the created test cases from the first deliverable. Suite number references are a count of the total number of tests created. They can also be used as references to find the specific test in our source code under tests/services/.

placeOrder:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
1	SA-ORD-01	productID=101 quantity=10	Valid product, valid quantity.	Success=true, orderId and amountDue returned	Pass
2	SA-ORD-02	productID=200, quantity=1	Product does not exist in inventory.	Success=false, error returned	Pass
3	SA-ORDER-05	productID=100, quantity = 0	Boundary value – quantity of zero.	success=false	Pass

4	SA-ORD-06	productID=100, quantity = -1	Negative quantity.	success=false	Pass
5	SA-ORD-03	productID = -1, quantity = 1	Invalid product ID.	success=false	Pass
6	SA-ORD-04	productID=100, quantity = 1,000,000	Extremely large quantity, depends on stock.	Outcome is Boolean and both paths are handled.	Pass
7	Extra test created that was not specified in deliverable 1.	productID = 101, quantity = 2	Checks if correct payload shape is sent to /orders	apiClient.post called with { items } array.	Pass

trackOrderProgress:

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
8	SA-ORD-07	orderId=order- 1 (valid, exists)	Is it a valid order in system?	Returns status string e.g. 'accepted'	Pass
9	SA-ORD-08	orderId=99999 (does not exist)	Order not found.	Returns 'unknown'	Pass
10	SA-ORD-09	orderId=-1 (invalid)	Invalid order ID.	Returns 'unknown'	Pass
11	SA-ORD-10	orderId=0 (boundary)	Boundary value.	Returns 'unknown'	Pass

getOrderDetails (additional coverage and not in original test case specification):

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
12	N/A	Valid orderID	Full order object returned.	All fields present and correctly mapped to camelCase.	Pass
13	N/A	Non existent orderID	Order not found.	Throws error.	Pass
14	N/A	Valid orderID	Correct endpoint called.	GET /orders{id} called.	Pass

queryBalance:

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
15	SA-ORD-11	merchantID =1000 (valid, exists)	Valid merchant found	Returns balance with creditLimit, currentDebt, availableCredit.	Pass
16	SA-ORD-12	merchantID=9999 (does not exist)	Merchant not found	Throws error.	Pass
17	SA-ORD-13	merchantID = -1 (invalid)	Invalid merchant ID	Throws error.	Pass.
18	SA-ORD-14	merchantID = 0 (boundary)	Boundary value	Throws error.	Pass.

viewPreviousOrders:

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
19	SA-ORD-15	merchantID = 1000 , has orders	Merchant exists and has placed orders.	Returns list of order summaries.	Pass
20	SA-ORD-16	merchantID=1000 , has no orders.	Merchant exists but no orders placed.	Returns empty list.	Pass
21	SA-ORD-17	merchantID = 2000 (does not exist)	Merchant not found.	Throws error.	Pass.
22	SA-ORD-18	merchantID = -1 (invalid)	Invalid merchant ID	Throws error.	Pass.

getCatalogue:

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
23	SA-ORD-19	none – catalogue already populated by demo script.	Product exists in catalogue.	Returns list of products with all fields.	Pass
24	SA-ORD-20	None – catalogue is empty.	No products in catalogue.	Returns empty list.	Pass
25-27	N/A	Various	General endpoint verification and error handling.	Correct endpoint called; returns [] on error.	Pass.

orderService unit test result: 27/27 passed.

authService.js (LoginAPIImpl – Test Cases 28-56):

merchantLogin:

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
28	SA-LOG-01	username = "merchant1", password = "correctPassword".	Valid credentials.	success = true, user returned, JWT stored via a setAuthToken	Pass
29	SA-LOG-02	username = "merchant1", password = "wrongPassword".	Wrong password.	success = false, error returned.	Pass
30	SA-LOG-03	username = "unknown", password = "pass".	Merchant does not exist.	success = false, error returned.	Pass
31	SA-LOG-04	username = "", password = "pass"	Empty username.	success = false.	Pass.
32	SA-LOG-05	username = "merchant1", Password = "".	Empty password.	success = false.	Pass.
33	SA-LOG-06	username = "", password = "".	Both fields empty.	success = false.	Pass
34-36	N/A	Various.	Token absence, token clearing, payload shape.	success = false when no token, clearAuthToken called, correct body sent.	Pass

merchantDisconnect:

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
37	SA-LOG-07	(valid session)	Merchant logged in, logout requested.	Returns true and token cleared.	Pass
38	SA-LOG-08/09	(backend errors)	Session not found or merchant does not exist.	Returns false and token still cleared locally.	Pass
39	SA-LOG-10	-	Correct logout endpoint hit.	POST /auth/logout called.	Pass.

Additional staff management functions (getCurrentUser, changePassword, getStaffUsers, createStaffUser, changeUserRole, deleteStaffUser – suites 40-56)

authService unit test result: 29/29 passed.

Overall unit test results: 56/56 passed.

Section 7.2 Subsystem and System Testing

7.2.1 Subsystem Testing – Provided Interface Tests

IPOS-SA provides two interfaces used by external subsystems:

ISALoginAPI is used by IPOS-CA to authenticate merchants.

ISAOOrderAPI is used by IPOS-CA to place and track orders.

ISALoginAPI.merchantLogin (Suites 57-68):

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
57	SA-LOG-01	username = "merchant1", password = "correctPass".	Valid credentials.	success = true, user object returned.	Pass
58	SA-LOG-02	username = "merchant1", password = "wrongPass".	Wrong password.	success = false, error returned.	Pass
59	SA-LOG-03	username = "unknown", password = "pass".	Merchant does not exist.	success = false, error returned.	Pass
60	SA-LOG-04	username = "", password = "pass"	Empty username.	success = false, error returned.	Pass.
61	SA-LOG-05	username = "merchant1", Password = "".	Empty password.	success = false, error returned.	Pass.
62	SA-LOG-06	username = "", password = "".	Both fields empty.	success = false, error returned.	Pass
63-64	N/A	Various.	HTTP interface verification: POST to /auth/login, correct	All interface contract conditions met.	Pass

			body fields, JWT stored on success, not stored on failure, cleared before each attempt. Missing token should also be treated as failure.		
--	--	--	--	--	--

Result: 12/12 passed.

ISAOrderAPI.placeOrder (Suites 69-78):

Suite	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
69	SA-ORD-01	productID = 1001, quantity = 5	Valid product, valid quantity.	success = true.	Pass
70	SA-ORD-02	productID = 9999, quantity = 5	Product does not exist.	success = false.	Pass
71	SA-ORD-05	productID = 1001, quantity = 0	Boundary value – zero quantity.	success = false.	Pass
72	SA-ORD-06	productID = 1001, quantity = -1	Negative quantity.	success = false.	Pass.
73	SA-ORD-03	productID = -1, quantity = 5	Invalid product ID.	success = false.	Pass.
74	SA-ORD-04	productID = 1001, quantity = 1000000	Extremely large quantity.	Outcome is boolean regardless of stock level.	Pass
75-78	N/A	Various.	HTTP Interface verification:	All interface contract	Pass

			POST to /orders, items array in body, snake_case to camelCase mapping of response. Check if network error returns failure object and does not throw.	conditions met.	
--	--	--	--	-----------------	--

Result: 10/10 passed.

Overall subsystem test result: 22/22 passed.

7.2.2 System Testing – Required Interface

IPOS-SA uses IPUCommsAPI provided by IPOS-PU to send email notifications to applicants following a commercial membership decision. The method under test is `sendEmail(email, body, subject)`. This is accessed via the `notifyApplication` wrapper in `notificationService.js` in `frontend/services/`.

Results are not required and can only be obtained during integration testing with the IPOS-PU team.

`notifyApplication.test.js`:

Suite	Test Case ID	Parameter Values	Description	Expected Outcome
79	SA-IPU-01	Email = application@test.com , valid outcome = 'approved', valid company ref.	All valid parameters.	Returns true. POST made to IPOS-PU with correct email, correct subject containing outcome, as well as a body containing the company reference.
80	SA-IPU-03	Outcome = 'pending' (non-terminal state).	Pending – no email should be sent.	Returns false without calling <code>sendEmail</code> .
81	SA-IPU-04/05	IPOS-PU unavailable (connection refused)	Required interface unavailable.	Returns false. An attempt was made but did not crash.

82	SA-IPU-06	email = "", outcome = 'pending'.	Completely invalid input	Returns false without calling sendEmail.
----	-----------	----------------------------------	--------------------------	--